

Implementation Report — Student Performance Analytics

What was built

Piece	File	What it does
Metric engine	backend/utils/analytics.js	Pure functions for every formula — no DB access, testable offline
Report API	backend/routes/admin.js:591	GET /api/admin/students/:id/report — runs 6 SQL queries, feeds the engine
Admin UI	jsx	Knowledge ring + band, stat tiles, badges, charts, expected-time override
API method	frontend/src/api/index.js	adminAPI.getStudentReport(id, expectedMinutes)
Badge fix	backend/routes/scores.js:443	return → continue so students can earn more than one badge

Formulas implemented, with worked examples

1. Accuracy = (correct / total) × 100

Per unit and overall (overall uses **summed** counts, not averaged percentages, so a 40-question unit outweighs a 4-question one).

Example: 8 correct of 10 → **80%**

2. First Attempt Accuracy = (correct on first attempt / total questions) × 100

"First attempt" = the student's **earliest answer to each question** (via ROW_NUMBER() OVER (PARTITION BY question ORDER BY completed_at)), not "first attempt of the quiz". This is the literal reading of your spec.

Example: first answer to 10 questions, 7 correct → **70%** Mastery badge: every question correct on its first attempt.

3. Speed Score = (expected time / actual time) × 100, capped at 100

Expected time = time_per_question × question count, **scaled by attempt count** so retries aren't punished (see "Caught during review" below). An admin can override the per-quiz budget via the Expected-time field.

Example: 10 questions × 30s × 3 attempts = **900s** expected; student took **800s** → $900/800 \times 100 = 112.5 \rightarrow$ **100** (capped, never above 100)

4. Cognitive Efficiency = Accuracy / Average Response Time

Higher = faster AND more accurate. Unitless ratio.

Example: Accuracy 80, avg response 12.4s → **6.45**

5. Retention = (Accuracy later / Accuracy initial) × 100

Per unit, comparing the earliest vs latest attempt's accuracy. Returns **null** when the student has fewer than 2 attempts.

Example: later 85%, initial 70% → **121.43** (an improving student scores above 100 — that's correct, only the composite gets clamped).

6. Time Utilization = actual time / expected time

Example: 800s actual, 900s expected → **0.89×** (slightly under budget).

7. Knowledge Score = 0.50·Acc + 0.20·FirstAtt + 0.15·Speed + 0.15·Retention

Your exact worked example, verified against the code:

Accuracy 80, FirstAtt 70, Speed 90, Retention 85 → $0.5 \times 80 + 0.2 \times 70 + 0.15 \times 90 + 0.15 \times 85$
= $40 + 14 + 13.5 + 12.75 = \mathbf{80.25 \rightarrow \text{Good}}$

When retention is unknown (fewer than 2 attempts): the retention term is dropped and the other weights rescale by $1/0.85$ so the score stays on a 0–100 scale and isn't silently depressed:

$0.588 \times 80 + 0.235 \times 70 + 0.176 \times 90 = \mathbf{79.41}$ — UI labels it "retention excluded"

8. Knowledge Level Classification

Score	Level	Color
≥ 90	Excellent	green
75–89	Good	green
60–74	Moderate	amber
50–59	Poor	orange
< 50	Very Poor	red

9. Leaderboard Score = 0.5·Accuracy + 0.3·Speed + 0.2·Completion

Example: Acc 80, Speed 90, Completion 40 → $40 + 27 + 8 = \mathbf{75}$

10. Completion & Engagement = (units completed / units available) × 100

Your spec lists "Engagement score" and "Completion score" with **identical formulas** (units completed / units available), so the API returns it once and the UI shows it once. "Units available" = distinct published units platform-wide; a unit is "completed" when **best score $\geq 75\%$** — the same rule the student-facing Units page already uses, so the two views agree.

Example: 4 units completed of 15 available $\rightarrow$ **26.67%**

11. Badges (computed on read, shown in the report)

Badge	Condition
Accuracy	overall accuracy $> 90\%$
Speed	speed score ≥ 90
Consistency	≥ 3 units at $\geq 75\%$ accuracy
Perfect Score	any attempt at 100%
Mastery	any unit fully correct on first attempt

Implemented differently than the spec — and why

1. **"Learning ability score = (.35x..." — omitted.** The formula is truncated in your spec, and it would have duplicated Knowledge Score (same inputs). You confirmed omission.
 2. **Time Utilization** — your line says (time taken)/60 minutes. A flat 60-minute budget doesn't scale: a 5-question quiz would always show 0.02. I used actual / expected where expected is the real per-quiz budget (time_per_question $\times$ question count), consistent with Speed Score. Same number family, but meaningful for short and long quizzes.
 3. **Badges are derived per-request, not stored.** The five report badges recompute from attempts on every view, so they're always correct and add no write path. The existing DB-backed achievements (Quiz Master, On Fire, ...) are separate — and the return $\rightarrow$ continue fix in Part D now makes those actually award multiple times.
 4. **Speed Score's expected time defaults to time_per_question** (the value you picked) and scales by attempt count, with an admin override stored in localStorage rather than a settings table — your choice, and zero DB impact.
-

Not implemented — and why

Item	Status	Reason
Live-game results in the metrics	✗ Not included	socket.js only ever writes live_sessions — live answers are never persisted to quiz_attempts/question_answers. Including them would need a persistence layer. The UI shows "Live-game sessions are not included" so the figures aren't mistaken for complete coverage.
Badges written to user_achievements	✗ Computed on read instead	Keeps them automatically correct and avoids a write path; can be persisted later if you want them on student profiles.
Leaderboard page reordered by Leaderboard Score	✗ Shown in report only	The live Leaderboard.jsx still ranks by XP. Re-sorting it is a product decision, out of scope.
login_streak / rank achievements	✗ Explicitly never awardable	Seeded in the DB but require daily-login tracking and a leaderboard snapshot that don't exist. Now earned = false with a comment instead of silently falling through.
Per-question response times listed in the report	⚠ Aggregated (avg/fastest/slowest)	Per-question times are already visible in the existing attempt drill-down table. Aggregates cover the report's purpose.

One bug caught during review before shipping

The expected-time query originally summed **actual** time over every attempt but budgeted **expected** time once per quiz — a student who retried a quiz 3× was scored 3× actual against 1× expected (e.g. 10q×30s×3 retries: correct Speed 100 vs the buggy **37.5**). Expected time now scales by attempt count, matching how actual time accumulates.

Verification status: build passes, module parses, and every formula above was run and matches (including your 80.25 → Good example). What's **not** yet verified is the SQL against real data — the joins, the ROW_NUMBER partitions, and the retention CTE only prove correct on execution, so give it a run on your staging DB: a student with ≥ 2 attempts in one unit (ring, band, badges, charts), a 1-attempt student (Retention "N/A", no NaN), and a 0-attempt student (clean empty state).